Machine Coding: Multi-level Comments View

Problem: Build a comment section with unlimited nested replies (like Reddit / YouTube / Hacker News threads): show comments as a tree, reply at any level, edit, delete, like, collapse threads and sort.

Tech stack: React (Vite, JavaScript) *(default, not given)*

Time limit: 60 minutes *(default, not given)*

PART A: Thought process

General steps for any machine coding task

1. **Understand the requirements** – functional, non-functional, ask clarifying questions, write down assumptions.
2. **List interactions / use cases** – every action the user can take and what should happen. This is the test checklist.
3. **Design the architecture** – components and their one-line responsibilities.
4. **Design state / data model** – what is stored, what is derived, who owns it.
5. **Design data structures and contracts** – sample data, API shape, mocks.
6. **Trace the data flow** – one interaction from click to pixels.
7. **Identify the hard parts** – solve them on paper before typing.
8. **Implement in layers** – simplest working version first, then add one working layer at a time.
9. **Cover states and edge cases** – loading, empty, error, and problem-specific cases.
10. **Wrap up** – explain trade-offs and what you would add with more time.

Time budget (60 minutes)

Time	Activity
0–8 min	Requirements, clarifying questions, assumptions
8–15 min	Data model (flat vs nested), component tree, reducer actions
15–40 min	Core build: render tree recursively → add reply → like/edit/delete
40–50 min	Collapse, sort, loading/error/empty states, validation
50–55 min	Styling (thread lines, indentation cap), accessibility pass
55–60 min	Walk through edge cases, trade-offs, what's next

Tip: think out loud

The interviewer grades the *reasoning*, not only the final UI. Say things like: *"I'll normalize the comments into a map, because adding a reply 6 levels deep in a nested tree means copying the whole path immutably."* If you run out of time, a clearly explained design with a working core beats a half-built feature list.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Show a list of top-level comments, each with **any number of nested replies** at **any depth**.
- Each comment shows author, relative time ("2 hours ago"), text, like count.
- **Add** a top-level comment and **reply** to any comment at any level.
- **Edit** and **delete** your own comments.
- **Like / unlike** a comment.
- **Collapse / expand** a thread (hide all replies under a comment, show "N replies hidden").
- **Sort** comments: newest, oldest, top (most liked).

Non-functional requirements

Area	What it means here
Performance	Adding a reply deep in the tree must not copy the whole tree; render cost stays proportional to visible comments.
Scalability	Works for hundreds of comments and 10+ levels; data shape can later support lazy-loaded replies.
Error handling	Loading / error / retry for the fetch; empty and whitespace-only text is blocked; length limit.
Accessibility	Real `<button>` s, `aria-expanded` on collapse/reply, `aria-pressed` on like, keyboard submit (Ctrl+Enter) and cancel (Esc), visible focus.
Reusability	One recursive `CommentItem` for every level; one `CommentForm` for new comment, reply and edit.
Extensibility	Reducer actions are a single place to plug in API calls, optimistic updates, real-time events.

Clarifying questions to ask

1. Is there a **depth limit**, or can replies nest forever? *(How should deep threads look?)*
2. When a comment **with replies** is deleted, do the replies disappear or stay?
3. Does the backend send a **flat list with** `parentId` or an already-nested tree?
4. Are replies loaded **all at once** or **lazily** ("load more replies")?
5. Who can edit/delete — only the author? Is there auth?
6. Do we need **real-time** updates (other users' comments appearing live)?

Assumptions we build with

- Unlimited depth; indentation stops growing after **5 levels** so the text keeps a readable width.
- Deleting a comment **with replies** = soft delete (shows "*This comment was deleted*", replies stay). Deleting a **leaf** removes it fully.

- The API returns a **flat list** with `parentId` (most common). We load everything at once from a **mock API** with a delay.
- Current user is hard-coded as `"you"`; only own comments show Edit/Delete.
- Writes (add, edit, like, delete) are local state only — no backend writes.
- Max comment length 500 characters.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page opens	"Loading comments…", then the tree renders
Fetch fails (`?fail` in URL)	Error message with **Retry** button
No comments	"No comments yet. Be the first to comment."
Type in top box + **Comment**	New top-level comment appears at the top (newest sort)
Empty / spaces only text	Submit button disabled
Click **Reply** on any comment	Inline reply box opens under that comment, focused
Submit reply	Reply appears nested under the parent; box closes; thread auto-expands
Esc in a reply/edit box	Box closes, nothing saved
Ctrl/⌘ + Enter	Submits the form
Click **▲ like**	Count +1, button highlighted; click again → −1
Click **Edit** (own comment)	Text becomes a textarea with current text; **Save** disabled until changed
Save edit	Text updates, "(edited)" label shown
Delete leaf comment (confirm)	Comment disappears
Delete comment with replies	Shows "[deleted] – This comment was deleted.", replies stay
Delete last reply of a soft-deleted comment	The empty deleted parent is removed too
Click **−** on a comment	All replies under it hide; shows "· N replies hidden"; button becomes **+**
Change **Sort by**	Every level re-orders (newest / oldest / top)
Reply nested 8 levels deep	Indentation stops after level 5; thread still readable

---

3. Design the architecture

Module	Responsibility
`App`	Owns the comments state (reducer), fetch status and sort; renders top form, toolbar and root comments.

Module	Responsibility
`CommentItem`	**Recursive.** Renders one comment + its actions + its replies (`CommentItem` again). Owns its own UI state: reply/edit mode and collapsed.
`CommentForm`	Controlled textarea used for new comment, reply and edit. Validation, char counter, keyboard shortcuts.
`commentsReducer`	Pure state transitions: loaded, add, edit, like, delete.
`utils/tree`	`normalize` flat list → map, `countReplies` , `sortIds` .
`services/api`	Mock `fetchComments()` with delay and failure mode.

```
App  (state: byId, rootIds, status, sortBy)
├── CommentForm               (new top-level comment)
├── Sort select
└── ul.comment-list
    └── CommentItem id=c1 depth=0     (state: mode, collapsed)
        ├── CommentForm               (reply / edit, when open)
        └── ul.replies
            ├── CommentItem id=c2 depth=1
            │   └── ul.replies
            │       └── CommentItem id=c3 depth=2 ...
            └── CommentItem id=c5 depth=1
```

Start simple: first version is just `App` + a recursive `CommentItem` rendering static data. `CommentForm` gets split out only when the same textarea logic is needed in three places (new, reply, edit).

---

4. Design the state / data model

State or field	Purpose
`byId`	Map `id → comment` (normalized). O(1) lookup and update of any comment at any depth.
`rootIds`	Ids of top-level comments.
`comment.childIds`	Ids of direct replies — this *is* the tree.
`comment.parentId`	Needed to remove a comment from its parent's `childIds` .
`comment.deleted`	Soft-delete flag for comments that still have replies.
`comment.liked / likes`	Like toggle for the current user + count.
`comment.editedAt`	Shows "(edited)".
`status` (App)	`loading` / `success` / `error` for the fetch.
`sortBy` (App)	`newest` / `oldest` / `top` .
`mode` (CommentItem)	`null` / `reply` / `edit` – which inline form is open.
`collapsed` (CommentItem)	Whether this comment's replies are hidden.

```
const initialState = { byId: {}, rootIds: [] };
const [state, dispatch] = useReducer(commentsReducer, initialState);
const [status, setStatus] = useState('loading');
const [sortBy, setSortBy] = useState('newest');

// inside CommentItem – pure UI state, local to each comment
const [mode, setMode] = useState(null);       // null | 'reply' | 'edit'
const [collapsed, setCollapsed] = useState(false);
```

Senior note – derive, don't store:

- **Sorted order** is derived at render (`sortIds`) — never stored, so a new like re-sorts "top" for free.
- **Reply count** ("5 replies hidden") is derived with `countReplies` — storing it means updating every ancestor on each add/delete.
- **Depth** is passed as a prop during recursion, not stored on the comment.
- **Total count** is derived from `byId` .
- **Ownership:** data (`byId` , `rootIds`) lives in `App` because many components change it. UI state (`mode` , `collapsed`) lives in each `CommentItem` because nobody else cares about it — collapsing one thread doesn't re-render the app.

---

5. Design the data structures and contracts

Why normalized (flat map) instead of a nested tree?

```
// Nested – natural to render, painful to update
[{ id: 'c1', text: '…', replies: [{ id: 'c2', replies: [{ id: 'c3', replies: [] }] }] }]

// Normalized – what we store
{
  byId: {
    c1: { id: 'c1', parentId: null, childIds: ['c2', 'c5'], … },
    c2: { id: 'c2', parentId: 'c1', childIds: ['c3'], … },
    c3: { id: 'c3', parentId: 'c2', childIds: [], … },
  },
  rootIds: ['c1', 'c6'],
}
```

To like `c3` in the nested version you must find it (DFS) and copy `c1 → c2 → c3` immutably. In the normalized version it's one spread: `{ ...byId, c3: { ...byId.c3, likes: 4 } }` . Stable string ids are also safe React `key` s.

Example data (what the API returns – flat, with `parentId`)

```
{ id: 'c1', parentId: null, author: 'priya', text: 'How do you handle deeply nested replies…?',
  createdAt: '2026-11-01T08:00:00Z', likes: 12 }
{ id: 'c2', parentId: 'c1', author: 'rahul', text: 'Normalize it…', createdAt: '2026-11-01T08:30:00Z',
likes: 8 }
```

API contracts (mocked)

Endpoint	Request	Response
`GET /posts/:postId/comments`	–	`200 Comment[]` (flat list)
`POST /posts/:postId/comments`	`{ parentId, text }`	`201 Comment` (server id + createdAt)
`PATCH /comments/:id`	`{ text }`	`200 Comment` (with `editedAt`)
`DELETE /comments/:id`	–	`200 { id, deleted: true }` or `204`
`POST /comments/:id/like`	`{ liked: true }`	`200 { likes, liked }`

Only `GET` is mocked (`fetchComments`, 600 ms delay, `?fail` to force an error). Writes go straight to the reducer; the reducer actions are where real API calls plug in later.

Reducer actions (internal contract)

```
{ type: 'loaded', list }                      // flat list from API
{ type: 'add', comment: { id, parentId, author, text, createdAt } }
{ type: 'edit', id, text, editedAt }
{ type: 'like', id }
{ type: 'delete', id }
```

---

6. Trace the data flow

Interaction: user replies to comment `c3` (depth 2).

```
Click "Reply" on c3
        ↓
CommentItem(c3): setMode('reply')  → inline CommentForm renders, autofocused
        ↓
User types "Makes sense" and presses Ctrl+Enter
        ↓
CommentForm: trims, validates (not empty) → onSubmit('Makes sense')
        ↓
CommentItem(c3).handleReply → actions.add('c3', 'Makes sense')
        ↓
actions.add creates id (crypto.randomUUID) + createdAt → dispatch({ type: 'add', comment })
        ↓
commentsReducer:
   byId = { ...byId, [newId]: newComment }
   byId.c3 = { ...c3, childIds: [newId, ...c3.childIds] }      ← only 2 objects created
        ↓
App re-renders with new state.byId → CommentItem tree re-renders
        ↓
CommentItem(c3): setMode(null), setCollapsed(false)
        ↓
Output: reply box closes; new reply shows under c3 at depth 3 with a thread line
```

---

7. Important technical challenges

Challenge 1: Immutable updates at any depth

Problem: With a nested tree, liking a comment 7 levels deep means a recursive search plus copying all 7 ancestors (`{ ...c1, replies: [...c1.replies with c2 replaced …] }`). Easy to mutate by mistake, and every ancestor gets a new reference.

Solution: Normalize once on load. Every update becomes "copy one or two entries in a map".

```
export function normalize(list) {
  const byId = {};
  for (const c of list) byId[c.id] = { likes: 0, liked: false, deleted: false, editedAt: null, ...c,
childIds: [] };
  const rootIds = [];
  for (const c of list) {
    if (c.parentId && byId[c.parentId]) byId[c.parentId].childIds.push(c.id);
    else { byId[c.id].parentId = null; rootIds.push(c.id); } // orphan → top level
  }
  return { byId, rootIds };
}
```

Two passes so the order of the list doesn't matter (a reply can come before its parent). O(n).

Challenge 2: Recursive rendering

Problem: We don't know the depth in advance, so we can't write nested `map` s by hand.

Solution: A component that renders itself for each child id. The base case is a comment with no `childIds` . `depth` is passed down for styling only.

```
{hasReplies && !collapsed && (
  <ul className={depth < MAX_INDENT_DEPTH ? 'replies indented' : 'replies'}>
    {sortIds(childIds, byId, sortBy).map((childId) => (
      <CommentItem key={childId} id={childId} byId={byId} actions={actions}
                   sortBy={sortBy} depth={depth + 1} />
    ))}
  </ul>
)}
```

Using `<ul>/<li>` nesting gives screen readers the list structure ("list, 2 items, level 3"). Collapsing simply doesn't render the subtree, so hidden threads cost nothing.

Challenge 3: Deleting a comment that has replies

Problem: User deletes `c6` which has a reply `c7` . If we remove `c6` , `c7` is orphaned (lost or shown without context). If we cascade-delete, we destroy other people's replies.

Solution: Leaf → hard delete. Has replies → soft delete (`deleted: true` , text cleared). And when the **last** reply of a soft-deleted parent is removed, the parent is now an empty "[deleted]" shell → remove it too, recursively up the chain.

```
export function removeComment(state, id) {
  const comment = state.byId[id];
  if (!comment) return state;
  if (comment.childIds.length > 0) return update(state, id, { deleted: true, text: '' });

  const byId = { ...state.byId };
  delete byId[id];
  if (!comment.parentId) return { byId, rootIds: state.rootIds.filter((x) => x !== id) };
  const parent = byId[comment.parentId];
  byId[parent.id] = { ...parent, childIds: parent.childIds.filter((x) => x !== id) };
  const next = { byId, rootIds: state.rootIds };
  return parent.deleted && byId[parent.id].childIds.length === 0 ? removeComment(next, parent.id) :
next;
}
```

Challenge 4: Deep threads in the UI

Problem: With 16 px indent per level, level 20 leaves almost no width on mobile; text becomes one word per line.

Solution: Indent only while `depth < MAX_INDENT_DEPTH` (5). Deeper replies still nest logically but line up with level 5. Long words are wrapped with `overflow-wrap: anywhere`. (With more time: Reddit-style "*Continue this thread →*" link that opens the subtree as its own page.)

```
.replies.indented { margin-left: 12px; padding-left: 12px; border-left: 2px solid #e4e7eb; }
.comment-text { white-space: pre-wrap; overflow-wrap: anywhere; }
```

---

8. Implement in the correct order

#	Step	Working result
1	Seed data (flat list) + `normalize`	Data in `{ byId, rootIds }` shape (log it)
2	Recursive `CommentItem` rendering author, time, text	Static nested tree on screen
3	Reducer + `useReducer` in `App`, `loaded` action	Same tree, now from reducer state
4	Mock `fetchComments` + loading / error / retry	Realistic load with spinner text
5	`CommentForm` + top-level `add`	Can post a new comment
6	Reply: `mode` state in `CommentItem`, `add` with `parentId`	Reply at any depth
7	Like toggle	Counts update
8	Edit (reuse `CommentForm` with `initialValue`) + "(edited)"	Edit own comments
9	Delete with soft/hard rules + confirm	Deleted threads behave correctly
10	Collapse / expand + hidden reply count	Long threads manageable
11	Sort (derived `sortIds`)	Newest / oldest / top at every level
12	CSS thread lines, indent cap, focus styles, a11y attributes	Polished and accessible

9. Cover all states and edge cases

State / edge case	Handling
Initial / loading	`status = 'loading'` → "Loading comments…" with `role="status"`
Error	`role="alert"` message + **Retry** re-calls `load`
Empty	"No comments yet. Be the first to comment." (top form still visible)
Success	Tree rendered, sorted by `sortBy`
Empty / whitespace-only input	Submit disabled; text is trimmed before saving
Over 500 chars	`maxLength` on textarea + live counter
Edit with no change	Save disabled (`trimmed === initialValue`)
Reply to a comment deleted meanwhile	Reducer ignores `add` if parent missing or deleted
Delete comment with replies	Soft delete, replies stay
Delete last reply of a deleted parent	Parent removed too (recursive cleanup)
Orphan in API data (`parentId` not found)	Shown as a top-level comment instead of disappearing
Replies listed before parent in API data	Two-pass `normalize` handles any order
Very deep nesting	Indentation capped at 5 levels
Long words / URLs	`overflow-wrap: anywhere`
Line breaks in comment	`white-space: pre-wrap` keeps them; React escapes text (no XSS)
Like spam (double click)	Toggle, not increment — always consistent
Adding a comment while loading	Form only shown after success, so `loaded` can't overwrite it
Reply to a collapsed thread	Thread auto-expands so the user sees their reply

10. Final structure

```
_51_comment_view_multi_level_comments/
├── index.html
├── package.json
├── vite.config.js
└── src/
    ├── main.jsx                (unchanged Vite entry)
    ├── App.jsx                 state owner: reducer, status, sort
    ├── App.css
    ├── constants.js            CURRENT_USER, MAX_LENGTH, MAX_INDENT_DEPTH
    ├── data/
    │   └── seed.js             flat mock comments
    ├── services/
    │   └── api.js              fetchComments() mock
    ├── state/
```

```
│   └── commentsReducer.js    loaded / add / edit / like / delete
├── utils/
│   ├── tree.js               normalize, countReplies, sortIds
│   └── format.js             timeAgo
└── components/
    ├── CommentItem.jsx       recursive comment + replies
    └── CommentForm.jsx       new / reply / edit textarea
```

---

11. Full working code

`src/constants.js`

```
export const CURRENT_USER = 'you';
export const MAX_LENGTH = 500;
// Past this depth replies stop indenting, so deep threads keep a readable width.
export const MAX_INDENT_DEPTH = 5;
```

`src/data/seed.js`

```
// Flat list, the way most APIs/DBs return comments. parentId null = top-level comment.
export const SEED = [
  { id: 'c1', parentId: null, author: 'priya', text: 'How do you handle deeply nested replies without copying the whole tree?', createdAt: '2026-11-01T08:00:00Z', likes: 12 },
  { id: 'c2', parentId: 'c1', author: 'rahul', text: 'Normalize it: keep comments in a map by id and store only childIds on each node.', createdAt: '2026-11-01T08:30:00Z', likes: 8 },
  { id: 'c3', parentId: 'c2', author: 'priya', text: 'So adding a reply only touches the new comment and its parent?', createdAt: '2026-11-01T09:00:00Z', likes: 3 },
  { id: 'c4', parentId: 'c3', author: 'rahul', text: 'Exactly. Two object copies, no matter how deep the reply is.', createdAt: '2026-11-01T09:10:00Z', likes: 5 },
  { id: 'c5', parentId: 'c1', author: 'meera', text: 'Also cap the indentation, or level 10 becomes a 40px wide column.', createdAt: '2026-11-01T10:00:00Z', likes: 6 },
  { id: 'c6', parentId: null, author: 'arjun', text: 'Should deleting a comment delete all of its replies too?', createdAt: '2026-11-01T11:00:00Z', likes: 4 },
  { id: 'c7', parentId: 'c6', author: 'you', text: 'Reddit keeps the replies and shows [deleted] in place of the parent.', createdAt: '2026-11-01T11:20:00Z', likes: 2 },
];
```

`src/services/api.js`

```
import { SEED } from '../data/seed';

const DELAY = 600;

// Mock of GET /posts/:postId/comments. Open the app with ?fail to see the error state.
export function fetchComments() {
  const fail = new URLSearchParams(window.location.search).has('fail');
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (fail) reject(new Error('Could not load comments'));
      else resolve(structuredClone(SEED));
    }, DELAY);
```

```
  });
}
```

`src/utils/tree.js`

```
// Flat list -> { byId, rootIds }. Each node keeps only the ids of its direct replies.
export function normalize(list) {
  const byId = {};
  for (const c of list) {
    byId[c.id] = { likes: 0, liked: false, deleted: false, editedAt: null, ...c, childIds: [] };
  }
  const rootIds = [];
  for (const c of list) {
    if (c.parentId && byId[c.parentId]) {
      byId[c.parentId].childIds.push(c.id);
    } else {
      byId[c.id].parentId = null; // orphan (parent missing) is shown as a top-level comment
      rootIds.push(c.id);
    }
  }
  return { byId, rootIds };
}

// Total replies under a comment at every level (shown when a thread is collapsed).
export function countReplies(byId, id) {
  return byId[id].childIds.reduce((sum, childId) => sum + 1 + countReplies(byId, childId), 0);
}

const COMPARATORS = {
  newest: (a, b) => b.createdAt.localeCompare(a.createdAt),
  oldest: (a, b) => a.createdAt.localeCompare(b.createdAt),
  top: (a, b) => b.likes - a.likes || b.createdAt.localeCompare(a.createdAt),
};

// Sorting is derived at render time; the stored order never changes.
export function sortIds(ids, byId, sortBy) {
  const cmp = COMPARATORS[sortBy];
  return [...ids].sort((a, b) => cmp(byId[a], byId[b]));
}
```

`src/utils/format.js`

```
const rtf = new Intl.RelativeTimeFormat('en', { numeric: 'auto' });
const UNITS = [
  ['year', 31536000],
  ['month', 2592000],
  ['day', 86400],
  ['hour', 3600],
  ['minute', 60],
];

export function timeAgo(iso, now = Date.now()) {
  const secs = Math.round((new Date(iso).getTime() - now) / 1000);
  for (const [unit, size] of UNITS) {
    if (Math.abs(secs) >= size) return rtf.format(Math.round(secs / size), unit);
  }
```

```
  return 'just now';
}
```

`src/state/commentsReducer.js`

```
import { normalize } from '../utils/tree';

export const initialState = { byId: {}, rootIds: [] };

function update(state, id, changes) {
  return { ...state, byId: { ...state.byId, [id]: { ...state.byId[id], ...changes } } };
}

// Leaf -> removed. Has replies -> soft delete so the replies keep their context.
// A soft-deleted parent whose last reply is removed has nothing left to show, so it goes too.
export function removeComment(state, id) {
  const comment = state.byId[id];
  if (!comment) return state;
  if (comment.childIds.length > 0) return update(state, id, { deleted: true, text: '' });

  const byId = { ...state.byId };
  delete byId[id];
  if (!comment.parentId) {
    return { byId, rootIds: state.rootIds.filter((x) => x !== id) };
  }
  const parent = byId[comment.parentId];
  byId[parent.id] = { ...parent, childIds: parent.childIds.filter((x) => x !== id) };
  const next = { byId, rootIds: state.rootIds };
  return parent.deleted && byId[parent.id].childIds.length === 0 ? removeComment(next, parent.id) :
next;
}

export function commentsReducer(state, action) {
  switch (action.type) {
    case 'loaded':
      return normalize(action.list);

    case 'add': {
      const c = { likes: 0, liked: false, deleted: false, editedAt: null, ...action.comment, childIds:
[] };
      const byId = { ...state.byId, [c.id]: c };
      if (!c.parentId) return { byId, rootIds: [c.id, ...state.rootIds] };
      const parent = byId[c.parentId];
      if (!parent || parent.deleted) return state; // parent vanished meanwhile
      byId[parent.id] = { ...parent, childIds: [c.id, ...parent.childIds] };
      return { ...state, byId };
    }

    case 'edit':
      if (!state.byId[action.id]) return state;
      return update(state, action.id, { text: action.text, editedAt: action.editedAt });

    case 'like': {
      const c = state.byId[action.id];
      if (!c) return state;
      return update(state, action.id, { liked: !c.liked, likes: c.likes + (c.liked ? -1 : 1) });
    }

    case 'delete':
```

```
      return removeComment(state, action.id);

    default:
      return state;
  }
}
```

`src/components/CommentForm.jsx`

```
import { useState } from 'react';
import { MAX_LENGTH } from '../constants';

export default function CommentForm({ initialValue = '', submitLabel, onSubmit, onCancel, autoFocus =
false }) {
  const [text, setText] = useState(initialValue);
  const trimmed = text.trim();
  // Empty, whitespace-only and unchanged edits can't be submitted.
  const canSubmit = trimmed !== '' && trimmed !== initialValue.trim();

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!canSubmit) return;
    onSubmit(trimmed);
    setText('');
  };

  const handleKeyDown = (e) => {
    if (e.key === 'Enter' && (e.ctrlKey || e.metaKey)) handleSubmit(e);
    if (e.key === 'Escape' && onCancel) onCancel();
  };

  return (
    <form className="comment-form" onSubmit={handleSubmit}>
      <textarea
        value={text}
        onChange={(e) => setText(e.target.value)}
        onKeyDown={handleKeyDown}
        maxLength={MAX_LENGTH}
        rows={3}
        placeholder="Write a comment..."
        aria-label={submitLabel}
        autoFocus={autoFocus}
      />
      <div className="form-row">
        <span className="hint">
          {text.length}/{MAX_LENGTH}
        </span>
        {onCancel && (
          <button type="button" onClick={onCancel}>
            Cancel
          </button>
        )}
        <button type="submit" disabled={!canSubmit}>
          {submitLabel}
        </button>
      </div>
    </form>
```

```
  );
}
```

`src/components/CommentItem.jsx`

```
import { useState } from 'react';
import CommentForm from './CommentForm';
import { countReplies, sortIds } from '../utils/tree';
import { timeAgo } from '../utils/format';
import { CURRENT_USER, MAX_INDENT_DEPTH } from '../constants';

export default function CommentItem({ id, byId, actions, sortBy, depth = 0 }) {
  const [mode, setMode] = useState(null); // null | 'reply' | 'edit'
  const [collapsed, setCollapsed] = useState(false);

  const comment = byId[id];
  if (!comment) return null;

  const { author, text, createdAt, editedAt, likes, liked, deleted, childIds } = comment;
  const isOwner = author === CURRENT_USER && !deleted;
  const hasReplies = childIds.length > 0;

  const handleDelete = () => {
    if (window.confirm('Delete this comment?')) actions.remove(id);
  };

  const handleReply = (value) => {
    actions.add(id, value);
    setMode(null);
    setCollapsed(false); // show the reply that was just added
  };

  const handleEdit = (value) => {
    actions.edit(id, value);
    setMode(null);
  };

  return (
    <li className="comment">
      <article aria-label={deleted ? 'Deleted comment' : `Comment by ${author}`}>
        <header className="comment-meta">
          {hasReplies && (
            <button
              type="button"
              className="toggle"
              aria-expanded={!collapsed}
              aria-label={collapsed ? 'Expand replies' : 'Collapse replies'}
              onClick={() => setCollapsed((c) => !c)}
            >
              {collapsed ? '+' : '−'}
            </button>
          )}
          <strong>{deleted ? '[deleted]' : author}</strong>
          <time dateTime={createdAt} title={new Date(createdAt).toLocaleString()}>
            {timeAgo(createdAt)}
          </time>
          {editedAt && !deleted && <span className="muted">(edited)</span>}
          {collapsed && <span className="muted">· {countReplies(byId, id)} replies hidden</span>}
        </header>
```

```
        {mode === 'edit' ? (
          <CommentForm initialValue={text} submitLabel="Save" autoFocus onSubmit={handleEdit} onCancel=
{() => setMode(null)} />
        ) : (
          <p className={deleted ? 'comment-text muted' : 'comment-text'}>{deleted ? 'This comment was
deleted.' : text}</p>
        )}

        {!deleted && mode !== 'edit' && (
          <div className="comment-actions">
            <button type="button" aria-pressed={liked} aria-label={`Like, ${likes} likes`} onClick={()
=> actions.like(id)}>
              ▲ {likes}
            </button>
            <button type="button" aria-expanded={mode === 'reply'} onClick={() => setMode(mode ===
'reply' ? null : 'reply')}>
              Reply
            </button>
            {isOwner && (
              <button type="button" onClick={() => setMode('edit')}>
                Edit
              </button>
            )}
            {isOwner && (
              <button type="button" onClick={handleDelete}>
                Delete
              </button>
            )}
          </div>
        )}
      </article>

      {mode === 'reply' && (
        <div className="reply-form">
          <CommentForm submitLabel="Reply" autoFocus onSubmit={handleReply} onCancel={() =>
setMode(null)} />
        </div>
      )}

      {hasReplies && !collapsed && (
        <ul className={depth < MAX_INDENT_DEPTH ? 'replies indented' : 'replies'}>
          {sortIds(childIds, byId, sortBy).map((childId) => (
            <CommentItem key={childId} id={childId} byId={byId} actions={actions} sortBy={sortBy} depth=
{depth + 1} />
          ))}
        </ul>
      )}
    </li>
  );
}
```

`src/App.jsx`

```
import { useCallback, useEffect, useMemo, useReducer, useState } from 'react';
import CommentForm from './components/CommentForm';
import CommentItem from './components/CommentItem';
import { commentsReducer, initialState } from './state/commentsReducer';
import { fetchComments } from './services/api';
```

```
import { sortIds } from './utils/tree';
import { CURRENT_USER } from './constants';
import './App.css';

export default function App() {
  const [state, dispatch] = useReducer(commentsReducer, initialState);
  const [status, setStatus] = useState('loading'); // loading | success | error
  const [sortBy, setSortBy] = useState('newest');

  const load = useCallback(() => {
    setStatus('loading');
    fetchComments()
      .then((list) => {
        dispatch({ type: 'loaded', list });
        setStatus('success');
      })
      .catch(() => setStatus('error'));
  }, []);

  useEffect(() => {
    load();
  }, [load]);

  // ids and timestamps are created here so the reducer stays pure.
  const actions = useMemo(
    () => ({
      add: (parentId, text) =>
        dispatch({
          type: 'add',
          comment: { id: crypto.randomUUID(), parentId, author: CURRENT_USER, text, createdAt: new
Date().toISOString() },
        }),
      edit: (id, text) => dispatch({ type: 'edit', id, text, editedAt: new Date().toISOString() }),
      remove: (id) => dispatch({ type: 'delete', id }),
      like: (id) => dispatch({ type: 'like', id }),
    }),
    []
  );

  const total = Object.values(state.byId).filter((c) => !c.deleted).length;

  return (
    <main className="app">
      <h1>
        Comments <span className="muted">({total})</span>
      </h1>

      {status === 'loading' && <p role="status">Loading comments…</p>}

      {status === 'error' && (
        <p role="alert" className="error">
          Could not load comments.{' '}
          <button type="button" onClick={load}>
            Retry
          </button>
        </p>
      )}

      {status === 'success' && (
        <>
          <CommentForm submitLabel="Comment" onSubmit={(text) => actions.add(null, text)} />
```

```
          <div className="toolbar">
            <label>
              Sort by{' '}
              <select value={sortBy} onChange={(e) => setSortBy(e.target.value)}>
                <option value="newest">Newest</option>
                <option value="oldest">Oldest</option>
                <option value="top">Top</option>
              </select>
            </label>
          </div>

          {state.rootIds.length === 0 ? (
            <p className="muted">No comments yet. Be the first to comment.</p>
          ) : (
            <ul className="comment-list">
              {sortIds(state.rootIds, state.byId, sortBy).map((id) => (
                <CommentItem key={id} id={id} byId={state.byId} actions={actions} sortBy={sortBy} />
              ))}
            </ul>
          )}
        </>
      )}
    </main>
  );
}
```

`src/App.css`

```
:root {
  font-family: system-ui, -apple-system, 'Segoe UI', sans-serif;
  color: #1f2933;
  background: #f5f7fa;
}
* { box-sizing: border-box; }
body { margin: 0; }
ul { list-style: none; margin: 0; padding: 0; }

.app { max-width: 720px; margin: 0 auto; padding: 24px 16px; }
h1 { font-size: 1.5rem; }
.muted { color: #7b8794; }
.error { color: #b42318; }

.toolbar { display: flex; justify-content: flex-end; margin: 16px 0 8px; font-size: 0.875rem; }

.comment-list > .comment {
  background: #fff;
  border: 1px solid #e4e7eb;
  border-radius: 8px;
  padding: 12px;
  margin-bottom: 12px;
}
.comment-meta { display: flex; align-items: center; gap: 8px; font-size: 0.875rem; }
.comment-text { margin: 4px 0 6px; white-space: pre-wrap; overflow-wrap: anywhere; }
.comment-actions { display: flex; gap: 4px; }

.comment-actions button,
.toggle {
  background: none;
```

```
  border: none;
  padding: 4px 8px;
  border-radius: 4px;
  color: #52606d;
  font-size: 0.8125rem;
  cursor: pointer;
}
.comment-actions button:hover,
.toggle:hover { background: #eef2f7; }
.comment-actions button[aria-pressed='true'] { color: #e3550f; font-weight: 600; }
.toggle { width: 24px; padding: 2px 0; font-weight: 700; }

/* the left border is the "thread line" */
.replies.indented { margin-left: 12px; padding-left: 12px; border-left: 2px solid #e4e7eb; }
.reply-form { margin: 4px 0 8px 24px; }

.comment-form textarea {
  width: 100%;
  padding: 8px;
  border: 1px solid #cbd2d9;
  border-radius: 6px;
  font: inherit;
  resize: vertical;
}
.form-row { display: flex; align-items: center; justify-content: flex-end; gap: 8px; margin-top: 6px; }
.form-row .hint { margin-right: auto; font-size: 0.75rem; color: #7b8794; }
.form-row button { padding: 6px 14px; border: 1px solid #cbd2d9; border-radius: 6px; background: #fff;
cursor: pointer; }
.form-row button[type='submit'] { background: #2563eb; border-color: #2563eb; color: #fff; }
.form-row button:disabled { opacity: 0.5; cursor: not-allowed; }

button:focus-visible,
textarea:focus-visible,
select:focus-visible { outline: 2px solid #2563eb; outline-offset: 2px; }
```

`src/main.jsx` stays as generated by Vite (renders `<App />` in `StrictMode`); remove the template `index.css` import or keep it empty.

---

12. Wrap-up

Edge cases handled

Loading / error + retry / empty states, whitespace-only and over-length input, unchanged edits, soft vs hard delete with recursive cleanup, orphans and out-of-order API data, replying to a deleted parent, deep nesting indent cap, long words, auto-expand after replying to a collapsed thread, keyboard submit/cancel.

Trade-offs to mention

- **Normalized vs nested:** normalized makes updates O(1) and safe; the cost is a `normalize` step and passing `byId` around. Worth it for anything editable.
- **Props vs Context:** `byId` and `actions` are passed as props through the recursion — explicit and easy to follow. Context would remove the drilling but still re-render every consumer on each change.

- **Re-render scope:** any change creates a new `byId`, so all *visible* `CommentItem`s re-render. Fine for hundreds of comments. For thousands: `React.memo(CommentItem)` + pass the `comment` object itself, or a store with per-id selectors (Redux/Zustand) so only the changed comment re-renders.
- **Soft delete** keeps context but leaves "[deleted]" placeholders; cascade delete is simpler but destroys other users' replies.
- **Local UI state** (`mode`, `collapsed`) per comment: simple and isolated; downside is it resets if the comment unmounts (e.g. parent collapsed).
- **Sorting at render** is O(k log k) per level on every render; cheap for typical thread sizes, could be memoized.
- **Writes are local only** – no optimistic update / rollback yet.

What to add with more time

- Real API writes with **optimistic updates + rollback** on failure (temp id replaced by server id).
- **Lazy-load replies** ("Load 12 more replies") and pagination of root comments.
- **"Continue this thread →"** for depth > N.
- **Virtualization** (react-window) for very long threads.
- Real-time updates via WebSocket / SSE, merged through the same reducer.
- Persist to `localStorage`; markdown / mentions; reporting.
- Tests: reducer unit tests (add/delete cascade/like), RTL tests for reply + collapse flows, a11y checks.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why did you assume the API returns a flat list and not a nested tree?

Databases store comments as rows with a `parent_id`, so flat lists are the most common response. It also lets the server paginate. If the API returns a nested tree, I'd flatten it once (DFS) and use the same normalized store.

Q2. What if the interviewer says "max 3 levels of nesting"?

Two options: hide the **Reply** button at depth 3, or allow it but attach the reply to the depth-3 ancestor (YouTube style: everything after level 1 is flat with "@user" mentions). The data model doesn't change — only `parentId` chosen when adding.

State design

Q3. Why `useReducer` instead of `useState`?

There are 5 related transitions over the same data (load, add, edit, like, delete) with non-trivial logic (cascade delete). A reducer keeps them pure, in one file, unit-testable without React, and makes it easy to replay server events later.

Q4. Why is `collapsed` local state and not in the reducer?

It's UI state only one component cares about. Keeping it local means collapsing a thread doesn't create a new `byId` or re-render the whole tree. I'd lift it only if we needed "collapse all" or to persist it.

Q5. Why not store `replyCount` on each comment?

It's derivable. Storing it means updating every ancestor on each add/delete — a classic source of drift bugs. I compute it with `countReplies` only when a thread is collapsed.

Q6. Why do ids and timestamps get created in `actions.add` and not in the reducer?

Reducers must be pure: the same input gives the same output. `crypto.randomUUID()` and `new Date()` are side effects; in StrictMode a reducer can run twice and would create two different ids.

Hard challenges

Q7. Walk me through liking a comment 10 levels deep in a nested structure vs yours.

Nested: find the path (DFS, O(n)), then rebuild 10 objects and 10 arrays immutably. Normalized: `byId[id]` lookup O(1), one spread for the map and one for the comment. Same result, far less code and no risk of a missed copy.

Q8. Can recursion blow the stack with very deep threads?

Rendering 1000 levels deep could be slow and React does recurse, but realistic threads are < 50 levels. Real products cap visual depth and use "Continue this thread", which also bounds the recursion. `countReplies` could be made iterative with an explicit stack if needed.

Q9. What happens if a user deletes a comment while someone else is replying to it?

Locally, the reducer's `add` ignores a reply whose parent is missing or deleted. With a real backend, the server returns 404/409; we'd show "This comment was deleted" and keep the user's draft so their text isn't lost.

Q10. Why soft delete instead of deleting the whole subtree?

Replies belong to other users and may be valuable on their own. Soft delete keeps the conversation readable. We still clean up empty "[deleted]" shells when their last reply goes.

Implementation steps

Q11. Why reuse one `CommentForm` for new, reply and edit?

Same behavior in all three: validation, trim, char counter, Ctrl+Enter, Esc. Differences are just props (`initialValue`, `submitLabel`, `onCancel`). One place to fix bugs.

Q12. Why is the top-level form hidden while loading?

If the user posts while the fetch is in flight, the `loaded` action would replace the whole state and wipe their comment. Hiding the form until success avoids the race. Alternative: merge loaded data with local pending comments.

Q13. How is the comment text protected against XSS?

React escapes text children, so `<script>` is shown as text. We never use `dangerouslySetInnerHTML`. If we add markdown, we'd render it through a sanitizer (DOMPurify) or a safe markdown renderer.

Performance and scale

Q14. What if the post has 10,000 comments?

Don't load them all: paginate root comments and load replies lazily (`GET /comments/:id/replies?cursor=`), show "Load more replies". Render with `React.memo(CommentItem)` and per-id selectors so a like re-renders one row. For huge flat views, virtualize with `react-window` (variable heights make it harder for trees: flatten visible nodes into a list with depth first).

Q15. Every like re-renders all comments. How do you fix it?

Pass the `comment` object (not `byId`) to `CommentItem` and wrap it in `React.memo`; unchanged comments keep their reference in the normalized map, so memo skips them. Children ids come from `comment.childIds`, and each child reads its own comment via a selector (`useSelector(s => s.byId[id])` in Redux/Zustand). Now only the liked comment and nothing else re-renders.

Q16. What if the API is slow when posting a reply?

Optimistic update: add the reply immediately with a temp id and a "sending…" style, call the API, then replace the temp id with the server id on success or remove it + show "Failed, retry" (keeping the text) on error. Disable double submits while pending.

Trade-offs / "why not X?"

Q17. Why not Context API instead of passing props down the recursion?

Context removes prop drilling but every consumer re-renders on any change to the context value — same as now, just less visible. For this size, explicit props are clearer. For scale, a store with selectors beats both.

Q18. Why not use `index` as `key`?

Sorting and deleting change positions; index keys would make React reuse the wrong `CommentItem`, so local state (open reply box, collapsed) would jump to a different comment. Stable ids keep state attached to the right comment.

Q19. How would you add real-time comments from other users?

Subscribe via WebSocket/SSE and dispatch the same `add` / `edit` / `delete` actions. Dedupe by id (our own comment may come back over the socket). Optionally buffer them behind a "3 new comments" banner so the list doesn't jump while reading.

Q20. How would you test this?

Reducer unit tests: add at depth, like toggle, soft delete, cascade cleanup, orphan normalize. React Testing Library: reply flow (click Reply → type → submit → nested text appears), collapse shows "N replies hidden", Esc cancels. Plus an axe check for accessibility.